

UNIT 1 · WEEK 1

Introduction to operating systems

📅 Week 1 · dates to be confirmed </> Lab TD0 (solved + AI challenge) ~ 4 animations

LAB TD0 · SOLVED

Setting up the systems-programming environment

Before programming against the operating system you need four things: knowing how to read the manual, a program skeleton, a small utility library and a `Makefile` that builds everything with warnings turned into errors.

</> How to use the solved labs

Every lab comes with its complete solution. The code you see here is exactly the one in the `tds/` folder of the course, built with `-Wall -Wextra -Werror` and tested by `tds/tests/run.sh`; the outputs come from a real run. Your job: build, run, change something, predict what will happen and check it. **You always compile with `gcc`**, with warnings turned into errors: this is the line for each program (the `Makefile` uses exactly the same one to build them all at once).

Bash

```
cd tds/td0
gcc -std=gnu11 -O2 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -I../common
errors.c ../common/utils.c -o errors -lm
./errors

make -C tds          # the same gcc line for every program of every lab
(make prints each one)
tds/tests/run.sh td0 # tests one lab and prepares its working folder
tds/tests/tmp/td0
```

Optional: the terminal inside the hub

You can work on the labs with a **real terminal of your machine** inside this page: every recorded output that can be repeated has a ► **Run** button that runs the same command on your machine, and the ► **_ Terminal** button (bottom right, on lab pages) opens a terminal where you type anything. A browser cannot start processes by itself, so a small program of the course acts as a bridge.

Bash

```
make -C tds && tds/tests/run.sh    # once: builds every lab and prepares
tds/tests/tmp/td0 ... td8
python3 tds/terminal.py           # leave this terminal open while you work
```

The bridge prints a **link** (open it and the hub connects by itself) and a **key** like `8765.1a2b...`, which you can also paste in the panel. The key changes every time you start it. **Ctrl+C** closes it, together with all its terminals.

- **Always with gcc** : ► **Run** types, one command per line, `cd` to the lab's folder, one `gcc` per program in the command (the executable goes to the working folder, `tds/tests/tmp/td3`) and the command of the recorded output, which runs there, next to the test files. If you change the code and press it again, you see the effect; if `gcc` reports an error, nothing runs. ⚙ **Compile with gcc**, on each exercise's code, only compiles (with all its `-D...` variants). If the lab has no working folder yet, it first prepares it with `tests/run.sh`; **Prepare TDn** does just that. Compare your output with the recorded one: PIDs, addresses and times change, and explaining why is part of the lab.
- **Floating windows**: each terminal floats over the page while you read the statement, the manual or the animations. Move it by dragging its bar (or with the arrow keys when the bar has focus), resize it from the corner and minimize it with `-`; it remembers where you left it. `+` opens another one (up to 3), handy for a server and its client (TD2, TD7); ► **Run** types into the last one you used. 🗑 moves it to a separate window, always on top even of other apps, like the lab's PDF (recent Firefox and Chrome); closing it brings it back to the page.
- **Ubuntu**: start it in the Terminal (Ctrl+Alt+T), inside the course folder. The Firefox that comes with Ubuntu or Chrome both work. **Windows**: start it inside WSL 2 (the Ubuntu terminal), where you build the labs; the Windows browser reaches WSL's `127.0.0.1` because WSL forwards `localhost` by default.
- **Security**: a shell within reach of the browser is delicate, so the bridge only listens on `127.0.0.1` (never on the network), requires that session's key, rejects pages from other sites, opens at most 3 terminals and closes each one when you close the tab. It is *your* shell, with your permissions: do not paste commands you do not understand.

Without the bridge everything stays the same: copy the commands to your terminal. Outputs that need a separate server (TD7) or the `agentd` daemon (TD8) have no button; their steps are in the statement.

EXERCISES

- | | | | |
|---|--|---|---------------------------------|
| 1 | Manual pages and sections | 4 | Building with <code>make</code> |
| 2 | The program skeleton | 5 | Explore your operating system |
| 3 | The <code>libutils</code> library and <code>exit_if</code> | | |

TD0 · 1

● Solved

Manual pages and sections

Statement. In this course we write `cmd(k)` to say that `cmd` is documented in section *k* of the manual. Use `man -k` to search by keyword and answer: in which section is the C library `printf`? What are the other pages that show up? What is the difference between sections `3` and `3p`?

Solution

Bash

```
man -k printf | grep ^printf      # keyword search, keep the names that start
with printf
man 3 printf                      # the C library function
man 1 printf                      # the shell command with the same name
man 3p printf                    # the POSIX standard text (package
manpages-posix-dev)
```

SECTION	CONTENTS	EXAMPLE
1	Programs and shell commands	<code>ls(1)</code> , <code>printf(1)</code>
2	System calls (entries into the kernel)	<code>write(2)</code> , <code>fork(2)</code>
3	Library functions (glibc on Linux)	<code>printf(3)</code> , <code>perror(3)</code>
3p	The same function according to the POSIX standard: what must hold on <i>any</i> Unix	<code>printf(3p)</code>
5 · 7	File formats · overviews	<code>passwd(5)</code> , <code>signal(7)</code>

Answer. The C `printf` is in section 3; `printf(1)` is the shell command. Section 3 describes the implementation you have (glibc, with its extensions); 3p describes the portable contract. When you write for several systems, trust 3p; when you debug on Linux, read 3.

TD0 · 2

● Solved

The program skeleton

Statement. Every exercise starts from an empty program that includes the usual headers and `utils.h`. Copy it under a new name to start each exercise.

Solution



C tds/td0/skeleton.c

```
/*
 * TD0 - skeleton.c: the empty program every exercise starts from.
 * Copy it with a new name (cp td0/skeleton.c td1/echo.c) and build with
 * `make`.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    (void)argc;    /* silences -Wunused-parameter until you use them */
    (void)argv;
    return EXIT_SUCCESS;
}
```

The `(void)argc;` lines exist because we build with `-Wextra`, which warns about unused parameters, and with `-Werror`, which turns that warning into an error. Once the program uses `argc` and `argv`, delete them.

TD0 · 3

● Solved · tested

The `libutils` library and `exit_if`

Statement. Read `utils.h`. What does `exit_if` do and why does it use `errno`? Write a program that opens a file and uses `exit_if` to report the error if it does not exist.

Solution

C tds/common/utils.h

```
/*
 * utils.h - small helper library shared by every lab (libutils.a).
 * Fundamentals of Operating Systems - Universidad Tecnológica Indoamerica.
 * Based on the PG204 labs (ENSEIRB-MatMeca, F. Morandat / M. Faverge).
 */
```

```

#ifndef FSO_UTILS_H
#define FSO_UTILS_H

#include <signal.h>      /* sigset_t */
#include <stddef.h>      /* size_t */
#include <sys/types.h>   /* ssize_t, pid_t */

/*
 * Exit with status 1 if `condition` is true.
 * The message is `prefix: <strerror(errno)>` when errno is set, `prefix`
 otherwise.
 * errno is only meaningful right after a failed call, so use it right after
 the call you test.
 */
void exit_if(int condition, const char *prefix);

/* Write exactly `len` bytes (loops over partial writes). Returns len, or -1
 on error. */
ssize_t write_all(int fd, const void *buf, size_t len);

/* Copy everything from fd_in to fd_out until end of file (read/write loop).
 */
void transfer(int fd_in, int fd_out);

/* Naive recursive Fibonacci: a way to burn CPU time (fibo(40) takes about a
 second). */
unsigned long long fibo(int n);

/*
 * Split argv at the first "--": what comes before goes to args1, what comes
 after to args2.
 * Both arrays end with NULL. Returns -1 if a side is empty or there is no "
 --".
 */
int split_args(char *argv[], char *args1[], char *args2[]);

/* Fork a child that sleeps `seconds` seconds and exits with status 0.
 Returns its pid. */
pid_t create_waiting_child(int seconds);

/* Name of a standard signal ("SIGUSR1"), using the numbering of the running
 system. */
const char *signame(int sig);

/* Print the standard signals that belong to `set` on fd (uses write, async-

```

```

    signal-safe). */
void print_sigset(int fd, const sigset_t *set);

/* Print the pending and the blocked signals of the calling process on fd. */
void print_signals(int fd);

/* TD1, exercise 8: the structure written and read in binary by
write_struct/read_struct. */
struct nopad {
    char c1;
    long l;
    char c2;
};

#endif /* FSO_UTILS_H */

```

C tds/td0/errors.c

```

/*
 * TD0 - errors.c: how exit_if() reports a failed system call.
 * open() fails, sets errno, and exit_if prints "open missing.txt: No such
 * file or directory".
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "missing.txt";
    int fd = open(path, O_RDONLY);

    exit_if(fd == -1, path);          /* prefix = the file name */
    printf("%s opened as file descriptor %d\n", path, fd);
    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}

```

Real output

```
$ ./errors
missing.txt: No such file or directory
[exit status 1]
```

Real output

```
$ ./errors notes.txt
notes.txt opened as file descriptor 3
```

When a system call fails it returns `-1` and stores the error code (`ENOENT`, `EACCES` ...) in the global variable `errno`. `perror` turns that code into text. That is why `exit_if` must be called **right after** the call being checked: any call in between could change `errno`.



What was fixed with respect to the inherited version

The signal-name table used the macOS numbering (on Linux, `signame(SIGUSR1)` returned another name); `fibonacci` had different types in `utils.h` and `utils.c`; `create_waiting_child` ignored its parameter; and writes did not check their result. The table is now built with the system macros and a `write_all` function repeats partial writes.

TD0 · 4

● Solved

Building with `make`

Statement. Explain the main rules of the course `Makefile`: how `libutils.a` is built, how each exercise is compiled and how several versions of the same program are obtained with `-D` macros.

Solution



Text tds/Makefile

```
# The course compiles with gcc. make's built-in default is CC=cc, which "CC
?= gcc" would not change.
ifeq ($(origin CC),default)
CC := gcc
endif
CFLAGS  ?= -O2 -g
CFLAGS += -std=gnu11 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -Icommon
LDFLAGS += -pthread
LDLIBS  += -Lbin -lutils -lm

TDS := td0 td1 td2 td3 td4 td5 td6 td7
SRCS := $(foreach t,$(TDS),$(wildcard $(t)/*.c))
BINS := $(patsubst %.c,bin/%,$(SRCS))

# Variants the lab asks to build with different macros (-D...).
VARIANTS := \
    bin/td3/buffer_1 bin/td3/buffer_2 bin/td3/buffer_3 bin/td3/buffer_4
bin/td3/buffer_5 \
    bin/td3/fork_pid_0 bin/td3/fork_pid_1 bin/td3/fork_pid_2 bin/td3/fork_pid_3
\
    bin/td4/mypipe_bug \
    bin/td5/map_none bin/td5/map_read bin/td5/map_write bin/td5/map_exec
bin/td5/map_ew \
    bin/td6/philosophers_deadlock bin/td5/shm_race bin/td6/peterson_fence

.PHONY: all test clean $(TDS) td8

all: bin/libutils.a $(BINS) $(VARIANTS) td8

$(TDS): bin/libutils.a
    @$(MAKE) --no-print-directory $(filter bin/$@/%, $(BINS) $(VARIANTS))

bin/libutils.a: common/utils.c common/utils.h
    @mkdir -p bin
    $(CC) $(CFLAGS) -c common/utils.c -o bin/utils.o
    $(AR) rcs $@ bin/utils.o
```

- `bin/libutils.a`: compiles `utils.c` into an object and archives it with `ar`. A `.a` file is a *static* library: the linker copies into each executable only the functions it uses.
- `bin/%: %.c` is a pattern rule: `bin/td1/echo` is built from `td1/echo.c`. `$<` is the first prerequisite and `$@` the target.

- `-Lbin -lutils` tells the linker where to look for the library and which one to use;
`-Icommon` tells the compiler where `utils.h` is.
- `-D_GNU_SOURCE` enables every function in glibc (POSIX and Linux extensions);
`-pthread` will be needed for threads (TD6).

TD0 · 5

● Solved · real output

Explore your operating system

Statement. Find out with commands: which kernel and which distribution you use, which is the first process (PID 1), how many kernel modules are loaded, how many processors you see and how long each boot stage took.

Solution (real output on the course machine)

Real output

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod |
tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) +
16.571s (userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

- `uname` shows the **kernel** (Linux 7.0); `/etc/os-release`, the **distribution** (Ubuntu 26.04): one kernel serves many distributions.
- PID 1 is `systemd`, the first process the kernel launches at boot; every other process descends from it.
- 258 loaded modules: although Linux is monolithic, its drivers are loaded and unloaded on the fly.
- `systemd-analyze` splits the boot into firmware, loader, kernel and user space: the kernel is the shortest stage. (It may be unavailable on WSL or in containers.)

These times feed animation A1.3 · The boot process →

Checklist

Generative AI challenge · Your own Makefile with tests

Goal. Write, with the help of an AI, a Makefile that builds every exercise of a lab into its own binary and adds a `test` target that runs each program with a known input.

Starting prompt. Use it in the tool you prefer (Claude, ChatGPT, Gemini, Copilot, Codex...) with the *solved* version of the lab as context. Ask for a plan first and review every step before accepting it.

Prompt

Act as a C systems-programming tutor. I have a folder with `skeleton.c`, `utils.c` and `utils.h` (library `libutils.a`). I want a Makefile that: (1) builds `libutils.a` once, (2) builds every `.c` in the folder into an executable with `-Wall -Wextra -Werror`, (3) has a `test` target that runs every program and compares its output with an `.expected` file. First explain the pattern rules (%) and the automatic variables (`$@`, `$<`, `$^`) you will use; then give me the Makefile commented line by line.

How to check the result

- Run `make` twice: the second time nothing is rebuilt.
- Touching `utils.h` (`touch common/utils.h`) rebuilds everything that depends on it.
- Break one expected output on purpose: `make test` must fail.

Declare your use of AI: tool, relevant prompts, what you checked yourself and what you fixed. What counts is that you can explain and verify every decision.